

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería de Software

MATERIAL DIDÁCTICO COMPLETO

SEMANA 4 — TEMA 5

Coordinación y Consistencia Distribuida

Asignatura: Aplicaciones Distribuidas

Período: 2026-2027 PPA | Fechas: 25-31 mayo 2026

Prof. Guerrero Ulloa G.C.

1. Confirmación de Temas — Semana 4

Según el Programa Analítico (Sílabo) de Aplicaciones Distribuidas, la Semana 4 (25-31 de mayo de 2026) corresponde a la Unidad 1: Generalidades de los Sistemas Distribuidos, y cubre el siguiente contenido temático:

Componente	Detalle
Tema 5	Coordinación y Consistencia Distribuida
Subtema 1	El problema de la consistencia (Modelo CAP)
Subtema 2	Algoritmos de coordinación (elección de líder: Bully, Ring)
Subtema 3	Transacciones distribuidas: Commit en Dos Fases (2PC)
Subtema 4	Sistemas de consenso: Paxos y Raft (introducción)

Resultado de aprendizaje: Analiza las características y principios de los sistemas distribuidos, evaluando sus ventajas, limitaciones y retos desde una perspectiva crítica e interdisciplinaria.

Estrategias metodológicas: Aprendizaje basado en estudios de casos, Aprendizaje cooperativo.

Habilidades blandas: Liderazgo, Resolución de problemas.

Actividades de la semana: Exposición grupal (Corte 1 — EV), Entrega TA-PFC-E1 (Propuesta y Análisis del Sistema Distribuido), Práctica Experimental N°1 (GA-SUM-01: Comunicación entre procesos con gRPC y Sockets TCP con Lamport).

2. Práctica de Clase — Algoritmo Bully

2.1 Objetivo

Implementar y comprender el Algoritmo Bully de elección de líder en un sistema distribuido simulado con 5 procesos concurrentes en Java. El programa completo se escribe en un solo archivo: BullyElection.java.

2.2 Fundamento Teórico

El Algoritmo Bully (García-Molina, 1982) es un protocolo de elección de líder donde el proceso activo con el identificador (ID) más alto se convierte en coordinador. Cuando un proceso detecta que el líder actual ha fallado (por ejemplo, al no recibir heartbeat — mensaje periódico que indica "estoy vivo"), inicia una elección enviando mensajes ELECTION a todos los procesos con ID superior. Si algún proceso con ID mayor responde con OK, el proceso iniciador se detiene. Si nadie responde dentro de un timeout, el proceso se proclama líder y envía COORDINATOR a todos los demás.

2.3 Requisitos Previos

JDK 16 o superior (se usa la característica "record" de Java 16). IDE como IntelliJ IDEA, Eclipse, VS Code con Extension Pack for Java, o simplemente la línea de comandos (javac + java).

2.4 Estructura del Programa

Todo el código está en un solo archivo BullyElection.java dentro del paquete `ec.edu.uteq.distribuidas.coordinacion`. Contiene:

- 1. Enum TipoMensaje** — define los 3 tipos de mensaje del algoritmo: ELECTION, OK, COORDINATOR.
- 2. Record Mensaje** — registro inmutable con campos: tipo, emisor, destino. (Un "record" en Java 16+ genera automáticamente constructor, getters, equals, hashCode y toString.)
- 3. Clase interna Proceso** — implementa Runnable (interfaz estándar de `java.lang` para hilos). Cada instancia representa un nodo distribuido que se ejecuta como hilo independiente.
- 4. Método main()** — crea 5 procesos, simula la caída del líder P4, y verifica la elección automática.

2.5 Paquetes Java Utilizados

Todos los paquetes usados son parte del JDK estándar (no se requieren librerías externas):

Clase / Interfaz	Paquete y Propósito
<code>BlockingQueue<T></code>	<code>java.util.concurrent</code> — Cola thread-safe que bloquea al consumidor si está vacía. Usada como buzón de mensajes.
<code>LinkedBlockingQueue<T></code>	<code>java.util.concurrent</code> — Implementación enlazada de <code>BlockingQueue</code> . Capacidad ilimitada por defecto.
<code>CompletableFuture</code>	<code>java.util.concurrent</code> — Permite ejecutar tareas asíncronas. Usamos <code>delayedExecutor()</code> para simular timeouts.

AtomicInteger	java.util.concurrent.atomic — Entero que se puede leer/escribir de forma atómica (thread-safe) sin synchronized.
TimeUnit	java.util.concurrent — Enumeración que representa unidades de tiempo (MILLISECONDS, SECONDS, etc.).
Runnable	java.lang (implícito) — Interfaz funcional estándar para definir la tarea que ejecuta un hilo (Thread).

2.6 Código Fuente Completo — BullyElection.java

A continuación, se presenta el código fuente completo, compilable y ejecutable. Copiar íntegramente en un archivo llamado BullyElection.java.

```
package ec.edu.uteq.distribuidas.coordinacion;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Simulación del Algoritmo Bully de elección de líder.
 * 5 procesos concurrentes (P0..P4) se comunican por buzones.
 * Cada proceso es un hilo independiente.
 *
 * Paquetes Java usados:
 * - java.util.concurrent.BlockingQueue: cola thread-safe para mensajes.
 * - java.util.concurrent.LinkedBlockingQueue: implementación enlazada.
 * - java.util.concurrent.CompletableFuture: para timeouts asíncronos.
 * - java.util.concurrent.atomic.AtomicInteger: entero atómico thread-safe.
 */
public class BullyElection {

    // --- Tipos de mensaje del algoritmo ---
    enum TipoMensaje { ELECTION, OK, COORDINATOR }

    // --- Registro inmutable que representa un mensaje ---
    // (record es una característica de Java 16+)
    record Mensaje(TipoMensaje tipo, int emisor, int destino) {}

    // --- Clase que representa un proceso/nodo distribuido ---
    static class Proceso implements Runnable {
        final int id; // Identificador único del proceso
        final int totalProcesos; // Total de procesos en el sistema
    }
}
```

```

final BlockingQueue<Mensaje>[] buzones; // Array compartido de buzones
volatile boolean activo;                // Si el proceso está vivo
volatile int liderActual = -1;          // ID del líder conocido
volatile boolean enEleccion = false;    // Flag de elección en curso

// Líder global compartido (solo para verificación externa)
static final AtomicInteger liderGlobal = new AtomicInteger(-1);

/**
 * Constructor.
 * @param id      Identificador del proceso (0..n-1)
 * @param total   Número total de procesos
 * @param buzones Array compartido de colas de mensajes
 * @param activo  Estado inicial (true=vivo, false=caído)
 */
Proceso(int id, int total, BlockingQueue<Mensaje>[] buzones, boolean activo) {
    this.id = id;
    this.totalProcesos = total;
    this.buzones = buzones;
    this.activo = activo;
}

/**
 * Bucle principal: espera mensajes en su buzón y los procesa.
 * poll() espera hasta 500ms por un mensaje; si no llega, vuelve a verificar.
 */
@Override
public void run() {
    System.out.printf("[P%d] Proceso iniciado (activo=%b)%n", id, activo);
    while (activo) {
        try {
            Mensaje msg = buzones[id].poll(500, TimeUnit.MILLISECONDS);
            if (msg != null) {
                procesarMensaje(msg);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }
    System.out.printf("[P%d] Proceso terminado%n", id);
}

/**
 * Procesa un mensaje recibido según su tipo.
 *
 * ELECTION: otro proceso con ID menor inició elección.
 */

```

```

*   -> Respondo OK (yo soy más grande) e inicio mi propia elección.
*
* OK: un proceso con ID mayor me respondió.
*   -> Me detengo; alguien más grande se encargará.
*
* COORDINATOR: un proceso se proclamó líder.
*   -> Actualizo mi referencia al nuevo líder.
*/
void procesarMensaje(Mensaje msg) {
    switch (msg.tipo()) {
        case ELECTION -> {
            System.out.printf("[P%d] Recibí ELECTION de P%d → respondo OK%n",
                               id, msg.emisor());
            enviar(TipoMensaje.OK, msg.emisor());
            if (!enEleccion) {
                iniciarEleccion();
            }
        }
        case OK -> {
            System.out.printf("[P%d] Recibí OK de P%d → me detengo%n",
                               id, msg.emisor());
            enEleccion = false;
        }
        case COORDINATOR -> {
            liderActual = msg.emisor();
            liderGlobal.set(liderActual);
            enEleccion = false;
            System.out.printf("[P%d] Nuevo líder reconocido: P%d%n",
                               id, liderActual);
        }
    }
}

/**
 * Inicia una elección Bully:
 * 1. Envía ELECTION a todos los procesos con ID mayor.
 * 2. Si nadie responde OK en 1500ms, se proclama líder.
 */
void iniciarEleccion() {
    enEleccion = true;
    System.out.printf("[P%d] *** Iniciando elección ***%n", id);
    boolean hayMasGrande = false;

    for (int i = id + 1; i < totalProcesos; i++) {
        enviar(TipoMensaje.ELECTION, i);
        hayMasGrande = true;
    }
}

```

```

        if (!hayMasGrande) {
            // Soy el de mayor ID activo → me proclamo líder
            proclamarLider();
        } else {
            // Espero 1500ms por respuestas OK
            CompletableFuture.delayedExecutor(1500, TimeUnit.MILLISECONDS)
                .execute(() -> {
                    if (enEleccion) {
                        // Nadie respondió → me proclamo líder
                        proclamarLider();
                    }
                });
        }
    }

    /**
     * Se proclama líder y envía COORDINATOR a todos los demás.
     */
    void proclamarLider() {
        liderActual = id;
        liderGlobal.set(id);
        enEleccion = false;
        System.out.printf("[P%d] ===== ME PROCLAMO LÍDER =====%n", id);

        for (int i = 0; i < totalProcesos; i++) {
            if (i != id) {
                enviar(TipoMensaje.COORDINATOR, i);
            }
        }
    }

    /**
     * Envía un mensaje al buzón del proceso destino.
     * Si el destino no está activo, el mensaje simplemente se ignora
     * (simula que el nodo no responde).
     */
    void enviar(TipoMensaje tipo, int destino) {
        Mensaje msg = new Mensaje(tipo, id, destino);
        buzones[destino].offer(msg);
    }
}

// ===== MAIN =====
@SuppressWarnings("unchecked")
public static void main(String[] args) throws InterruptedException {
    final int N = 5; // Número de procesos (P0..P4)

```

```

// Paso 1: Crear un buzón (BlockingQueue) para cada proceso
BlockingQueue<Mensaje>[] buzones = new LinkedBlockingQueue[N];
for (int i = 0; i < N; i++) {
    buzones[i] = new LinkedBlockingQueue<>();
}

// Paso 2: Crear los 5 procesos (todos activos inicialmente)
Proceso[] procesos = new Proceso[N];
Thread[] hilos = new Thread[N];
for (int i = 0; i < N; i++) {
    procesos[i] = new Proceso(i, N, buzones, true);
    hilos[i] = new Thread(procesos[i], "P" + i);
    hilos[i].start();
}

// Paso 3: P4 se proclama líder inicial (tiene el ID más alto)
System.out.println("\n--- ESTADO INICIAL: P4 se proclama líder ---");
procesos[4].proclamarLider();
Thread.sleep(1000); // Esperar a que todos reciban COORDINATOR

// Paso 4: Simular fallo de P4
System.out.println("\n--- SIMULANDO FALLO DE P4 ---");
procesos[4].activo = false;
Thread.sleep(500);

// Paso 5: P1 detecta el fallo e inicia elección
System.out.println("\n--- P1 DETECTA FALLO E INICIA ELECCIÓN ---");
procesos[1].iniciarEleccion();

// Paso 6: Esperar a que la elección se complete
Thread.sleep(3000);

// Paso 7: Verificar resultado
System.out.println("\n--- RESULTADO FINAL ---");
System.out.printf("Líder global: P%d\n", Proceso.liderGlobal.get());
System.out.println("Resultado esperado: P3 (el proceso activo con ID más alto)");

// Terminar todos los procesos
for (int i = 0; i < N; i++) {
    procesos[i].activo = false;
}
for (int i = 0; i < N; i++) {
    hilos[i].join(2000);
}
}
}

```


2.7 Compilar y Ejecutar

Desde la terminal, ejecutar los siguientes comandos:

```
mkdir -p ec/edu/uteq/distribuidas/coordinacion
cp BullyElection.java ec/edu/uteq/distribuidas/coordinacion/
javac ec/edu/uteq/distribuidas/coordinacion/BullyElection.java
java ec.edu.uteq.distribuidas.coordinacion.BullyElection
```

2.8 Salida Esperada

La salida mostrará la siguiente secuencia de eventos:

```
--- ESTADO INICIAL: P4 se proclama líder ---
[P4] ===== ME PROCLAMO LÍDER =====
[P0] Nuevo líder reconocido: P4
[P1] Nuevo líder reconocido: P4
[P2] Nuevo líder reconocido: P4
[P3] Nuevo líder reconocido: P4

--- SIMULANDO FALLO DE P4 ---
[P4] Proceso terminado

--- P1 DETECTA FALLO E INICIA ELECCIÓN ---
[P1] *** Iniciando elección ***
[P2] Recibí ELECTION de P1 → respondo OK
[P3] Recibí ELECTION de P1 → respondo OK
[P1] Recibí OK de P2 → me detengo
[P2] *** Iniciando elección ***
[P3] Recibí ELECTION de P2 → respondo OK
[P3] *** Iniciando elección ***
[P3] ===== ME PROCLAMO LÍDER =====

--- RESULTADO FINAL ---
Líder global: P3
```

El resultado confirma que P3 se convierte en líder, ya que es el proceso activo con el ID más alto (P4 está caído).

3. Práctica de Clase — Protocolo 2PC

3.1 Objetivo

Implementar y comprender el protocolo Two-Phase Commit (2PC — Confirmación en Dos Fases) simulando una transacción distribuida entre tres bases de datos geográficamente distribuidas (Quevedo, Guayaquil, Quito). El programa completo se escribe en un solo archivo: `Protocolo2PC.java`.

3.2 Fundamento Teórico

El protocolo 2PC garantiza atomicidad en transacciones distribuidas: o todos los participantes confirman (commit) o todos abortan (rollback). El coordinador dirige dos fases: FASE 1 — PREPARE (recolectar votos de cada participante) y FASE 2 — DECISION (enviar GLOBAL-COMMIT si todos votaron COMMIT, o GLOBAL-ABORT si alguno votó ABORT). WAL significa Write-Ahead Log (Registro de Escritura Anticipada): un archivo en disco donde se registran los cambios ANTES de aplicarlos, para poder deshacerlos si es necesario.

3.3 Estructura del Programa

Todo el código está en `Protocolo2PC.java` dentro del paquete `ec.edu.uteq.distribuidas.coordinacion`. Contiene:

- 1. Enums `VotoParticipante` y `DecisionGlobal`** — definen los posibles votos (COMMIT, ABORT) y decisiones globales (GLOBAL_COMMIT, GLOBAL_ABORT).
- 2. Interfaz `Participante`** — contrato que define los métodos que todo nodo participante debe implementar: `getNombre()`, `preparar()`, `aplicarDecision()`. NOTA: esta interfaz la definimos nosotros en el código; NO es una clase de la librería estándar de Java.
- 3. Clase `ParticipanteBD`** — implementa la interfaz `Participante`. Simula una base de datos con probabilidad de fallo configurable y un log interno (`ConcurrentHashMap`).
- 4. Clase `Coordinador`** — ejecuta el protocolo 2PC: envía PREPARE a todos, recolecta votos, toma la decisión global, y envía la decisión a todos.
- 5. Método `main()`** — ejecuta dos escenarios: Happy Path (todos votan COMMIT) y Fallo (BD-Quito siempre vota ABORT).

3.4 Paquetes Java Utilizados

Clase / Interfaz	Paquete y Propósito
<code>ConcurrentHashMap<K,V></code>	<code>java.util.concurrent</code> — Mapa thread-safe. Usado como log interno de cada participante.
<code>LinkedHashMap<K,V></code>	<code>java.util</code> — Mapa que preserva el orden de inserción. Usado para registrar votos en orden.
<code>List<T></code> , <code>ArrayList<T></code>	<code>java.util</code> — Lista dinámica. Usada para almacenar participantes y entradas de log.
<code>Random</code>	<code>java.util</code> — Generador de números aleatorios. Simula fallos con probabilidad configurable.
<code>Map<K,V></code>	<code>java.util</code> — Interfaz base de mapas. Usada como tipo de retorno del log.

3.5 Código Fuente Completo — Protocolo2PC.java

A continuación se presenta el código fuente completo, compilable y ejecutable. Copiar íntegramente en un archivo llamado Protocolo2PC.java.

```
package ec.edu.uteq.distribuidas.coordinacion;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Simulación del protocolo Two-Phase Commit (2PC).
 * Tres bases de datos geográficas (Quevedo, Guayaquil, Quito)
 * participan en transacciones coordinadas.
 *
 * Paquetes Java usados:
 * - java.util.concurrent.ConcurrentHashMap: mapa thread-safe para el log.
 * - java.util.LinkedHashMap: mapa que preserva orden de inserción.
 * - java.util.Random: generador de números aleatorios (simular fallos).
 *
 * Nota: "interface" en Java define un contrato (métodos que una clase
 * debe implementar). No es una clase de librería externa; la definimos
 * nosotros aquí dentro del archivo Protocolo2PC.java.
 */
public class Protocolo2PC {

    // --- Enumeraciones para los votos y decisiones ---
    enum VotoParticipante { COMMIT, ABORT }
    enum DecisionGlobal { GLOBAL_COMMIT, GLOBAL_ABORT }

    // =====
    // INTERFAZ: Participante
    // Define el contrato que todo nodo participante debe cumplir.
    // En Java, una "interface" es un tipo abstracto que especifica
    // métodos sin implementarlos. Las clases que la implementan
    // (con "implements") deben proveer el código de cada método.
    // =====
    interface Participante {
        /** Retorna el nombre identificador del nodo. */
        String getNombre();

        /**
         * FASE 1 del 2PC: el coordinador pide al participante que
         * PREPARE la transacción.
         */
    }
}
```

```

    * El participante escribe en su WAL (Write-Ahead Log) y vota:
    * - COMMIT si puede completar la operación.
    * - ABORT si detecta un problema.
    *
    * @param transaccionId Identificador único de la transacción.
    * @return VotoParticipante.COMMIT o VotoParticipante.ABORT
    */
    VotoParticipante preparar(String transaccionId);

    /**
     * FASE 2 del 2PC: el coordinador envía la decisión global.
     * Si es GLOBAL_COMMIT, el participante aplica los cambios.
     * Si es GLOBAL_ABORT, el participante deshace los cambios (rollback).
     *
     * @param decision GLOBAL_COMMIT o GLOBAL_ABORT
     * @param txId Identificador de la transacción
     * @return true si la decisión se aplicó correctamente
     */
    boolean aplicarDecision(DecisionGlobal decision, String txId);
}

// =====
// CLASE: ParticipanteBD
// Implementa la interfaz Participante simulando una base de datos.
// Tiene una probabilidad de fallo configurable y un log interno.
// =====
static class ParticipanteBD implements Participante {
    private final String nombre; // Ej: "BD-Quevedo"
    private final double probabilidadFallo; // 0.0 = nunca falla, 1.0 = siempre
    private final Random rng = new Random();
    // Log interno: registra cada acción por transacción
    // ConcurrentHashMap es de java.util.concurrent (thread-safe)
    private final Map<String, List<String>> log = new ConcurrentHashMap<>();

    /**
     * Constructor.
     * @param nombre Nombre del nodo (ej: "BD-Quevedo")
     * @param probabilidadFallo Probabilidad de votar ABORT (0.0 a 1.0)
     */
    public ParticipanteBD(String nombre, double probabilidadFallo) {
        this.nombre = nombre;
        this.probabilidadFallo = probabilidadFallo;
    }

    @Override
    public String getNombre() {
        return nombre;
    }
}

```

```

    }

    /**
     * FASE 1: Preparar la transacción.
     * Simula escritura en WAL (Write-Ahead Log).
     * Puede fallar aleatoriamente según probabilidadFallo.
     */
    @Override
    public VotoParticipante preparar(String txId) {
        // Simular posible fallo
        if (rng.nextDouble() < probabilidadFallo) {
            System.out.printf(" [%s] FASE 1: Fallo detectado → voto ABORT%n",
nombre);

            registrar(txId, "VOTE-ABORT (fallo simulado)");
            return VotoParticipante.ABORT;
        }

        // Escritura exitosa en WAL
        System.out.printf(" [%s] FASE 1: Escribí en WAL → voto COMMIT%n", nombre);
        registrar(txId, "PREPARED at " + System.currentTimeMillis());
        return VotoParticipante.COMMIT;
    }

    /**
     * FASE 2: Aplicar la decisión global.
     * GLOBAL_COMMIT → aplicar cambios desde WAL a la BD.
     * GLOBAL_ABORT → deshacer cambios (rollback).
     */
    @Override
    public boolean aplicarDecision(DecisionGlobal decision, String txId) {
        if (decision == DecisionGlobal.GLOBAL_COMMIT) {
            System.out.printf(" [%s] FASE 2: Recibí GLOBAL-COMMIT → cambios
aplicados%n",

                                nombre);
            registrar(txId, "COMMITTED at " + System.currentTimeMillis());
        } else {
            System.out.printf(" [%s] FASE 2: Recibí GLOBAL-ABORT → rollback
ejecutado%n",

                                nombre);
            registrar(txId, "ABORTED (rollback) at " + System.currentTimeMillis());
        }
        return true;
    }

    /** Registra una acción en el log interno. */
    private void registrar(String txId, String accion) {
        log.computeIfAbsent(txId, k -> new ArrayList<>()).add(accion);
    }

```

```

    /** Retorna el log completo para inspección. */
    public Map<String, List<String>> getLog() {
        return log;
    }
}

// =====
// CLASE: Coordinador
// Ejecuta el protocolo 2PC en dos fases.
// =====
static class Coordinador {
    private final List<Participante> participantes;

    /**
     * Constructor.
     * @param participantes Lista de nodos participantes en la transacción.
     */
    public Coordinador(List<Participante> participantes) {
        this.participantes = participantes;
    }

    /**
     * Ejecuta el protocolo 2PC completo para una transacción.
     *
     * @param txId Identificador único de la transacción.
     * @return true si la transacción se completó (GLOBAL-COMMIT),
     *         false si se abortó (GLOBAL-ABORT).
     */
    public boolean ejecutar(String txId) {
        System.out.printf("%n=== TRANSACCIÓN %s ===%n", txId);

        // ---- FASE 1: PREPARE - recolectar votos ----
        System.out.println("--- FASE 1: PREPARE (enviando a todos los participantes) -
--");

        Map<String, VotoParticipante> votos = new LinkedHashMap<>();
        for (Participante p : participantes) {
            VotoParticipante voto = p.preparar(txId);
            votos.put(p.getNombre(), voto);
        }

        // Mostrar resumen de votos
        System.out.println("--- Resumen de votos ---");
        votos.forEach((nombre, voto) ->
            System.out.printf(" %s → %s%n", nombre, voto));

        // ---- Determinar decisión global ----

```

```

        boolean todosCommit = votos.values().stream()
            .allMatch(v -> v == VotoParticipante.COMMIT);
        DecisionGlobal decision = todosCommit
            ? DecisionGlobal.GLOBAL_COMMIT
            : DecisionGlobal.GLOBAL_ABORT;

        System.out.printf("--- DECISIÓN: %s ---%n", decision);

        // ---- FASE 2: enviar decisión a todos ----
        System.out.println("--- FASE 2: Enviando decisión global ---");
        for (Participante p : participantes) {
            p.aplicarDecision(decision, txId);
        }

        System.out.printf("=== TRANSACCIÓN %s: %s ===%n%n", txId,
            todosCommit ? "COMPLETADA" : "ABORTADA");
        return todosCommit;
    }
}

// ===== MAIN =====
public static void main(String[] args) {
    // =====
    // ESCENARIO 1: Happy Path (todos votan COMMIT)
    // Tres BD con probabilidad de fallo 0.0 (nunca fallan)
    // =====
    System.out.println("=====");
    System.out.println("|| ESCENARIO 1: Happy Path (sin fallos) ||");
    System.out.println("=====");

    List<ParticipanteBD> participantes1 = List.of(
        new ParticipanteBD("BD-Quevedo", 0.0), // Nunca falla
        new ParticipanteBD("BD-Guayaquil", 0.0), // Nunca falla
        new ParticipanteBD("BD-Quito", 0.0) // Nunca falla
    );

    Coordinador coord1 = new Coordinador(new ArrayList<>(participantes1));
    boolean resultado1 = coord1.ejecutar("TX-001");

    // Imprimir logs de cada participante
    System.out.println("--- Logs internos (Escenario 1) ---");
    for (ParticipanteBD p : participantes1) {
        System.out.printf("  %s: %s%n", p.getNombre(), p.getLog());
    }

    // =====
    // ESCENARIO 2: BD-Quito siempre falla → ABORT

```

```

// =====
System.out.println("\n=====");
System.out.println("|| ESCENARIO 2: BD-Quito falla → ABORT ||");
System.out.println("=====");

List<ParticipanteBD> participantes2 = List.of(
    new ParticipanteBD("BD-Quevedo", 0.0), // Nunca falla
    new ParticipanteBD("BD-Guayaquil", 0.0), // Nunca falla
    new ParticipanteBD("BD-Quito", 1.0) // SIEMPRE falla
);

Coordinador coord2 = new Coordinador(new ArrayList<>(participantes2));
boolean resultado2 = coord2.ejecutar("TX-002");

// Imprimir logs
System.out.println("--- Logs internos (Escenario 2) ---");
for (ParticipanteBD p : participantes2) {
    System.out.printf(" %s: %s\n", p.getNombre(), p.getLog());
}

// =====
// VERIFICACIÓN FINAL
// =====
System.out.println("\n=====");
System.out.println("|| VERIFICACIÓN FINAL ||");
System.out.println("=====");
System.out.printf("TX-001 (Happy Path): %s\n",
    resultado1 ? "GLOBAL-COMMIT ✓" : "GLOBAL-ABORT X");
System.out.printf("TX-002 (Con fallo): %s\n",
    resultado2 ? "GLOBAL-COMMIT ✓" : "GLOBAL-ABORT X");
System.out.println("\nNota: En TX-001, los 3 logs muestran PREPARED +
COMMITTED.");
System.out.println("En TX-002, BD-Quito muestra VOTE-ABORT y todos muestran
ABORTED.");
}
}

```

3.6 Compilar y Ejecutar

```

mkdir -p ec/edu/uteq/distribuidas/coordinacion
cp Protocolo2PC.java ec/edu/uteq/distribuidas/coordinacion/
javac ec/edu/uteq/distribuidas/coordinacion/Protocolo2PC.java
java ec.edu.uteq.distribuidas.coordinacion.Protocolo2PC

```

3.7 Salida Esperada

Escenario 1 (Happy Path): las tres BD votan COMMIT → decisión GLOBAL-COMMIT → cambios aplicados en las tres. Escenario 2 (Fallo): BD-Quito vota ABORT → decisión GLOBAL-ABORT

→ rollback en las tres. Los logs internos confirman que en TX-001 todos tienen PREPARED + COMMITTED, y en TX-002 BD-Quito tiene VOTE-ABORT y todos tienen ABORTED.

4. Problema Propuesto — Para Resolver por los Estudiantes

4.1 Enunciado del Problema

Diseñar e implementar en Java un sistema distribuido de 3 nodos (N0, N1, N2) que simule un registro de inventario compartido con las siguientes funcionalidades:

Requisito 1: Elección de líder con Bully

Los 3 nodos usan el Algoritmo Bully para elegir un coordinador. Inicialmente, N2 (ID más alto) es el líder. Simular la caída de N2 y verificar que N1 se convierte en el nuevo líder automáticamente. Reutilizar la lógica de BullyElection.java adaptándola a 3 nodos.

Requisito 2: Transacciones con 2PC

El nodo coordinador (líder elegido por Bully) actúa como coordinador del protocolo 2PC. Cuando un nodo quiere escribir en el registro compartido (un `ConcurrentHashMap<String, Integer>` donde la clave es el nombre del producto y el valor es la cantidad), el coordinador ejecuta 2PC con los otros dos nodos como participantes. El método `preparar()` debe verificar si la operación es válida (por ejemplo, si hay suficiente stock para descontar). Implementar ambos casos: commit exitoso (todos votan COMMIT) y abort (un participante vota ABORT por stock insuficiente).

Requisito 3: Escenario completo

Ejecutar la siguiente secuencia y documentar la salida con capturas de pantalla:

1. N2 es líder. Ejecutar transacción TX-001 (registrar 100 unidades del producto A). Resultado esperado: COMMIT exitoso en los 3 nodos.
2. Simular fallo de N2. N1 detecta el fallo y se convierte en nuevo líder via Bully.
3. N1 como nuevo coordinador ejecuta TX-002 (descontar 30 unidades del producto A). Resultado esperado: COMMIT exitoso.
4. Ejecutar TX-003 (descontar 200 unidades del producto A, más de las disponibles). N0 detecta stock insuficiente y vota ABORT. Resultado esperado: GLOBAL-ABORT, inventario no cambia.

4.2 Criterios de Evaluación

Criterio	Peso	Puntos
El algoritmo Bully elige correctamente al líder tras la caída de N2	25%	2.5
El protocolo 2PC completa TX-001 y TX-002 con GLOBAL-COMMIT	25%	2.5
El protocolo 2PC aborta TX-003 correctamente con GLOBAL-ABORT	25%	2.5
El código compila, ejecuta y produce la salida esperada (con capturas)	25%	2.5

4.3 Pistas para la Implementación

Usar `BlockingQueue<Mensaje>` (de `java.util.concurrent`) para la comunicación entre nodos, igual que en `BullyElection.java`. El registro de inventario compartido puede ser un `ConcurrentHashMap<String, Integer>` (de `java.util.concurrent`) donde la clave es el nombre del

producto y el valor es la cantidad. En el método preparar() del participante 2PC, verificar si la operación es válida: para una operación de descuento, comprobar que el stock actual sea mayor o igual al descuento solicitado; si no, votar ABORT. Combinar la lógica de BullyElection y Protocolo2PC en un solo programa: el nodo elegido como líder por Bully es quien instancia el Coordinador del 2PC.

5. Práctica Experimental N°1 (GA-SUM-01)

Tema: Comunicación entre Procesos en Sistemas Distribuidos con gRPC y Sockets TCP

Semana de entrega: 4 (último día de la semana 4)

Código: GA-SUM-01 · Modalidad: Grupal (3-4 estudiantes) · Duración: 3h laboratorio + 5h autónomas

5.1 Objetivo General

Implementar y comparar mecanismos de comunicación entre procesos (sockets TCP y gRPC) en un sistema distribuido de 3 nodos, aplicando relojes lógicos de Lamport para ordenar eventos.

5.2 Procedimiento Paso a Paso

Paso 1: Configuración del Entorno

Instalar Python 3.10+ y las librerías necesarias: `pip install grpcio grpcio-tools protobuf matplotlib`. Crear repositorio GitHub con estructura: `/src/sockets/`, `/src/grpc/`, `/docs/`, `README.md`. Configurar 3 nodos (procesos en localhost con puertos 5000, 5001, 5002 o contenedores Docker). Cada integrante debe hacer al menos 3 commits significativos.

Paso 2: Implementación con Sockets TCP

Implementar servidor TCP usando `socket.socket` con `socket.SOCK_STREAM` (ambas clases del módulo estándar `socket` de Python, no requiere instalación). El servidor recibe mensajes JSON con la estructura `{"sender": str, "timestamp": int, "message": str}`. El cliente envía 100 mensajes y mide la latencia promedio de ida y vuelta (round-trip time). Aplicar relojes de Lamport: cada nodo mantiene un contador lógico entero que se incrementa con cada evento local y se actualiza al recibir un mensaje tomando `max(reloj_local, timestamp_recibido) + 1`.

Paso 3: Definir el servicio gRPC con Protocol Buffers

Crear el archivo `messaging.proto` con la definición del servicio `MessageService` que incluye el RPC `SendMessage(MessageRequest) returns (MessageReply)`. Generar el código Python con: `python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. messaging.proto`. Esto genera dos archivos automáticos: `messaging_pb2.py` (clases de datos) y `messaging_pb2_grpc.py` (stubs del servicio). Implementar el servidor y cliente gRPC equivalentes al sistema de sockets.

Paso 4: Ejecutar las pruebas de latencia

Enviar 100 mensajes con cada tecnología (sockets TCP y gRPC). Registrar la latencia de cada envío en archivos CSV (`latency_sockets.csv` y `latency_grpc.csv`). Generar boxplots comparativos con `matplotlib` (`import matplotlib.pyplot as plt`, del paquete `matplotlib` que se instaló en el Paso 1).

Paso 5: Documento LaTeX

Elaborar documento con fundamento teórico completo de la Unidad 1 (cada subtema con mínimo 300 palabras y al menos 2 referencias IEEE con DOI). Incluir código fuente dentro de bloques `lstlisting` (del paquete `listings` de LaTeX), figuras con `caption` y `label` (paquete `graphicx`), tablas con líneas horizontales formateadas (paquete `booktabs`), y bibliografía con mínimo 6 referencias IEEE verificadas. Compilar con `pdflatex` sin errores.

6. Problema Adicional — Extensión de la Práctica Experimental

6.1 Enunciado

Implementar un sistema de 5 nodos que simule una versión simplificada del algoritmo de elección de líder de Raft. Cada nodo tiene un estado (Follower, Candidate o Leader) y un término (term — número entero que se incrementa con cada elección). Las reglas son:

1. Un Follower que no recibe heartbeat del líder dentro de un timeout aleatorio (150-300ms) se convierte en Candidate e inicia una elección.
2. El Candidate incrementa su término, vota por sí mismo y envía RequestVote a todos los demás nodos (equivalente a un RPC — Remote Procedure Call).
3. Cada nodo solo puede votar una vez por término (term). Si el Candidate obtiene mayoría simple (3 de 5), se convierte en Leader.
4. El Leader envía heartbeats periódicos (cada 50ms) a todos los Followers para mantener su autoridad y evitar nuevas elecciones.

6.2 Requisitos Específicos

Usar `ScheduledExecutorService` (de `java.util.concurrent`, clase estándar del JDK) para simular los timeouts periódicos y los heartbeats. Usar `BlockingQueue<Mensaje>` (de `java.util.concurrent`) para los mensajes entre nodos. Probar con 5 nodos y simular 2 fallos simultáneos. Verificar que el sistema elige un nuevo líder correctamente tras los fallos. Documentar la salida con capturas que muestren las transiciones de estado (Follower→Candidate→Leader).

6.3 Rúbrica del Ejercicio Propuesto

Criterio	Peso	Pts
Transiciones de estado Follower → Candidate → Leader correctas	25%	2.5
Elección por mayoría con términos (terms) incrementales	25%	2.5
Heartbeat y timeout funcionales (<code>ScheduledExecutorService</code>)	25%	2.5
Tolerancia a 2 fallos simultáneos verificada con capturas	25%	2.5

7. Bibliografía

- [1] Brewer, E. (2012). CAP twelve years later: How the 'rules' have changed. IEEE Computer, 45(2), 23-29. doi: 10.1109/MC.2012.37.
- [2] van Steen, M. & Tanenbaum, A. S. (2023). Distributed Systems, 4th ed. Maarten van Steen.
- [3] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media. ISBN: 978-1449373320.
- [4] Ongaro, D. & Ousterhout, J. (2014). In Search of an Understandable Consensus Algorithm. Proc. USENIX ATC.
- [5] Lamport, L. (1998). The Part-Time Parliament. ACM TOCS, 16(2), 133-169.
- [6] Burns, B. (2025). Designing Distributed Systems, 2nd ed. O'Reilly Media.
- [7] Guerrero Ulloa, G. C. (2026). Aplicaciones Distribuidas — Material de apoyo. UTEQ. Páginas 62-74.